Bansilal Ramnath Agarwal Charitable Trust's
VISHWAKARMA INSTITUTE OF TECHNOLOGY, PUNE – 37
(An Autonomous Institute Affiliated to University of Pune)

DEEP LEARNING

COURSE PROJECT

Topic- Transformer-Powered Multi-Agent Framework for Adaptive Software Quality Engineering

DIV- CSAI-C **GROUP** - C2-03 **DAY**- TUESDAY **DATE**-29/09/2026

Roll no	Name	PRN
55	Kumar Kolhe	12410482
58	Radhyeshyam Lahoti	12410037
62	Akshada Lohakare	12411517
63	Divesh Lokhande	12412495

Name of Guide:

Dr. Anuradha Yenkikar

INTRODUCTION

Integrate structural, behavioral, visual and source-code evidence to create a traceable representation of a web application for intelligent quality engineering.

- Modern web applications are highly dynamic and contain complex user interfaces, asynchronous requests and interactive components.
- Traditional software testing primarily relies on DOM inspection, manually designed test cases and predefined validation rules.
- Single-source analysis cannot fully capture the application's structure, runtime behavior and rendered appearance.
- This project proposes a multi-modal deep learning-based analysis framework that combines web crawling, DOM, network, screenshot and repository evidence.
- OmniParser and Florence-2 enable visual GUI understanding, while evidence fusion provides richer information for downstream defect detection and test analysis.

MOTIVATION

Traditional testing falls short because DOM inspection misses visual bugs, while screenshots miss code structure.

- **Modern webs** are dynamic, using JavaScript, async APIs, and canvas elements that are hard to track.
- **Manual testing** is slow, making cross-page UI consistency checks time-consuming.
- **AI visual intelligence** is needed to let computer vision models read GUI regions and text.
- **Integrated evidence** is required, combining DOM, screenshots, and logs for better defect analysis.
- **Scalable execution** is essential, separating lightweight web crawling from heavy GPU AI inference.

Literature Review

Sr.NO.	Research Paper	Contribution	Limitation/ Gap	Relevance to AgentQE
1	Large Language Models for Software Testing: A Research Roadmap (2026)	Reviews LLM applications and challenges in software testing	Grounding, reliability, evaluation, and end-to-end integration remain open.	Base research for AgentQE's integrated AI testing approach.
2	Architectural Perspectives on GenAI-Driven Software Testing Systems: A Systematic Literature Review (2026)	Reviews GenAI testing architectures, pipelines, integration strategies and evaluation methods.	Existing systems show fragmentation, grounding and reproducibility challenges.	Closely supports AgentQE's integrated architecture: crawler → evidence → Knowledge Model → RAG → test generation.
3	Enhancing Large Language Models for Text-to-Testcase Generation(2025)	Studies prompting and fine-tuning for LLM-based test-case generation.	Mainly converts textual requirements into tests rather than using multimodal application evidence.	Directly supports, where AgentQE generates UI/API tests from requirements and retrieved application context.
4	OmniParser for Pure Vision Based GUI Agent (2024)	Parses screenshots into structured and actionable UI elements for GUI agents.	Visual parsing alone does not provide complete application, API or repository understanding.	Directly supports AgentQE's screenshot-based visual UI analysis and multimodal evidence collection.
5	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020)	Introduces RAG by combining retrieval with language-model generation.	General RAG is not designed specifically for software-application knowledge.	Foundation for AgentQE, where application Knowledge Model data is retrieved to ground test generation.

PROBLEM STATEMENT

To develop a modular, multi-modal deep learning framework that automatically explores web pages within configurable limits, collects DOM, network, screenshot and repository evidence, uses visual AI to identify and semantically interpret GUI elements, and correlates heterogeneous data to support defect analysis, automated test generation, validation and reporting.

Research Gap

Existing Approach	Identified Limitation
DOM-based analysis	Limited understanding of visual-only content
Screenshot-based analysis	Lacks HTML structure and source context
Network monitoring	Cannot independently interpret UI semantics
Standalone object detection	Identifies elements but may lack semantic understanding
Isolated testing evidence	Limited correlation across UI, DOM, network and code
Monolithic AI processing	Difficult to debug, replace models and trace results

OBJECTIVES

To develop a bounded multi-page web crawler with configurable limits, URL normalization and deduplication.

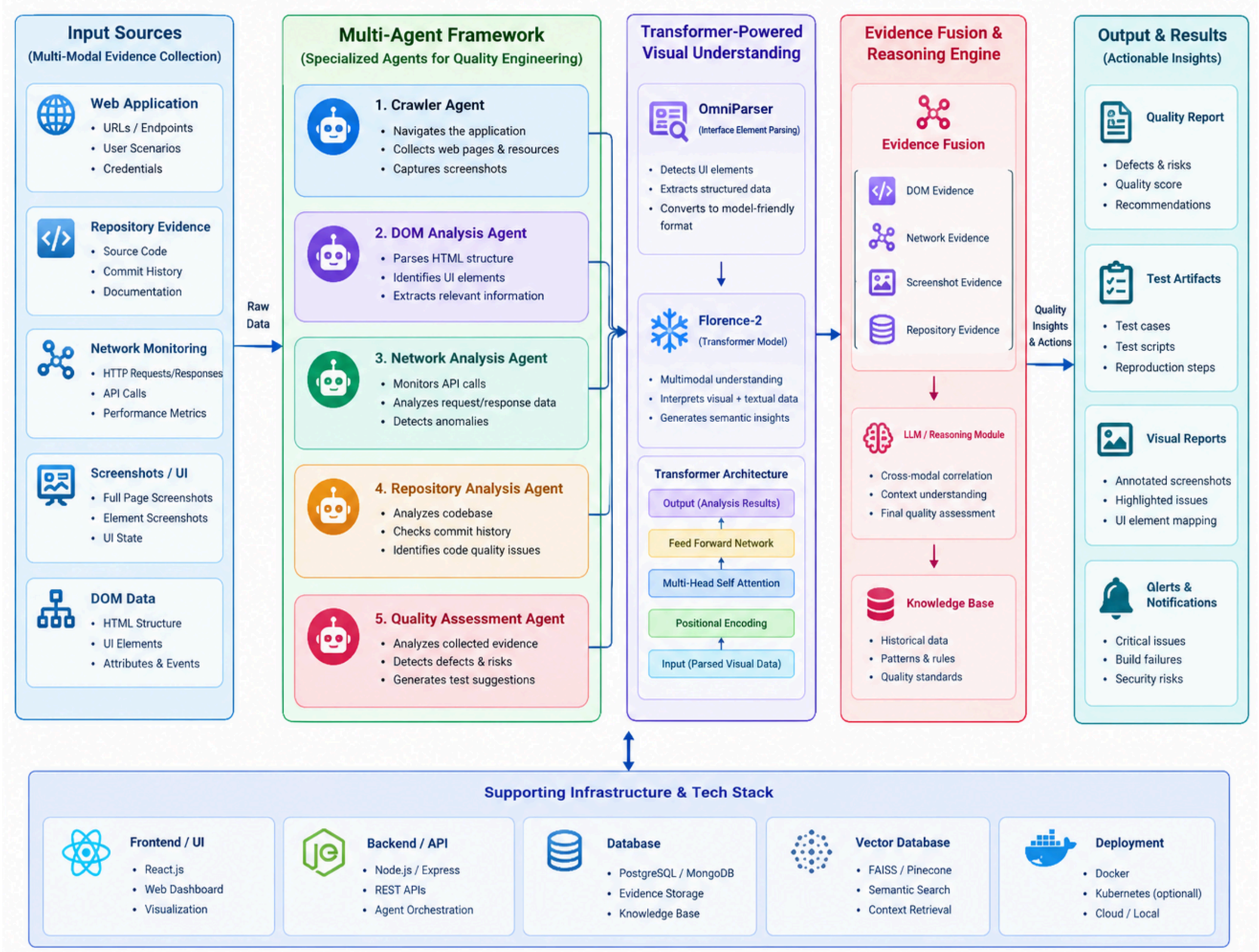
To capture synchronized DOM, network, screenshots, navigation and repository evidence.

To integrate OmniParser and Florence-2 for GUI element detection, OCR and semantic visual interpretation.

To implement multi-modal evidence fusion to correlate visual, DOM, network and source code information.

To enable automated defect analysis, test generation, validation, reporting and reproducible execution across CPU and GPU environments.

SYSTEM ARCHITECTURE



Tech Stack

Category	Technology	Purpose
Programming	Python	Crawler orchestration, AI integration and analysis
Browser	Chromium	Web rendering and runtime instrumentation
Browser Automation	Existing project crawler automation layer	Navigation and evidence collection
Deep Learning	PyTorch	Tensor operations and neural network inference
Transformer Framework	Hugging Face Transformers	Model loading, processing and generation
GUI Parsing	OmniParser	Visual GUI understanding
Object Detection	YOLO v9	GUI element localization
Vision-Language Model	Florence-2	Semantic visual understanding
OCR	EasyOCR / OCR Pipeline	Text extraction from screenshots
Image Processing	OpenCV and Pillow	Image preprocessing, cropping and annotation
Visualization	Supervision	Detection representation and visualization
Demo Interface	Gradio	Interactive visual inspection
Compute	CPU / CUDA GPU	Local inference and acceleration
Cloud Compute	Google Colab	Optional remote GPU execution
Storage	JSON, JSONL and Image Files	Structured evidence and reproducible artifacts

METHODOLOGY

Web Evidence Collection and Analysis Methodology

Trigger: a start URL and execution configuration are provided. End: traceable findings, test artifacts, evaluation datasets and report...

Input and Configuration

- Accept the start URL and define crawl limits
- Configure screenshot settings and model parameters
- Select the execution mode

Bounded Web Crawling

- Launch the browser and navigate to the start URL
- Normalize and deduplicate discovered URLs
- Enforce crawl scope and configured limits
- Discover additional links from in-scope pages

Multi-Modal Evidence Collection

- Capture DOM structure and page metadata
- Record network activity
- Capture screenshots for configured pages and states
- Collect available repository and source information

Visual AI Processing

- Preprocess captured screenshots
- Detect GUI regions with YOLO
- Extract visible text with OCR
- Semantically interpret regions with Florence-2 through the OmniParser pipeline

Structured Evidence Generation

- Generate visual JSON for each processed screenshot
- Add element coordinates and normalized bounding boxes
- Assign labels, extracted text and captions
- Store separate confidence values for detection, text extraction and interpretation
- Produce annotated screenshots

Evidence Fusion and Analysis

- Associate visual elements with corresponding DOM elements
- Correlate visual evidence with network and source evidence
- Extract features from the fused evidence
- Pass structured information to defect and test analysis

Validation and Reporting

- Generate findings and test artifacts
- Build evaluation datasets from traceable evidence
- Evaluate individual pipeline components
- Generate reports with evidence traceability
- Maintain provenance from outputs back to original evidence

MODELS USED

YOLO-based GUI Element Detector

OmniParser v2 uses YOLO-based detection to identify and localize GUI elements, generating bounding boxes, class predictions, confidence scores and spatial information.

OCR — EasyOCR / OCR Pipeline

Extracts visible text from screenshots, identifies text regions and confidence scores, and associates labels with nearby GUI elements.

Florence-2-Transformer-Based Vision-Language Model

Uses vision-language processing for semantic interpretation, captioning, object detection and OCR, supporting visual region and icon understanding in the project.

OmniParser v2 — Integrated GUI Parsing Pipeline

Combines visual detection, OCR and Florence-2 captioning to convert screenshots into structured GUI evidence with element regions and semantic descriptions for downstream fusion. It is a multi-stage pipeline, not a single model.

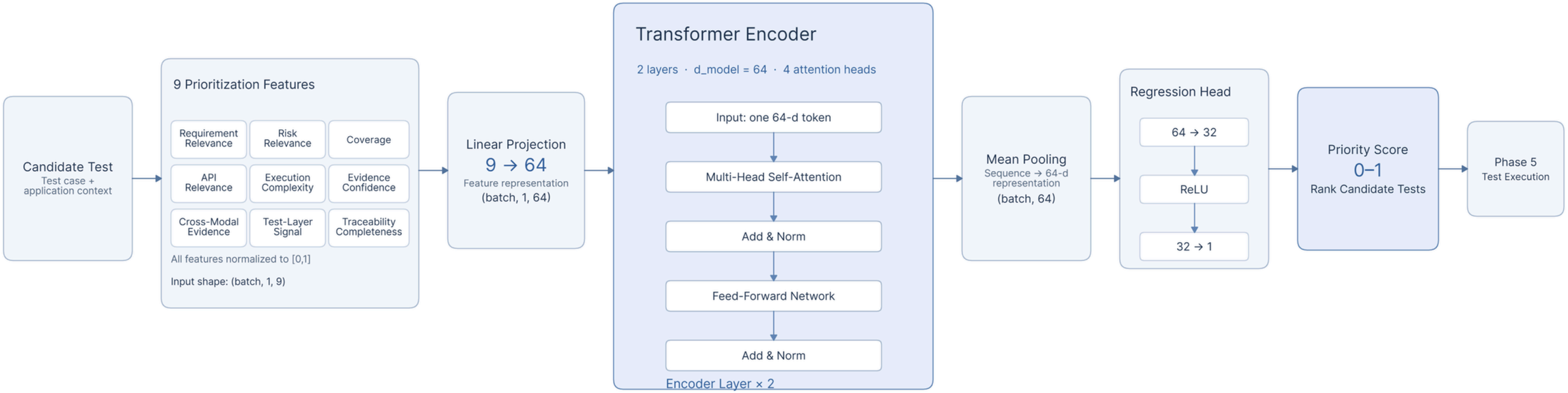
TRANSFORMER ARCHITECTURE

- **Tokenization & Embeddings**: Convert text into numerical representations.
- **Positional Information**: Captures token order.
- **Self-Attention**: Identifies relationships between tokens.
- **Multi-Head Attention**: Learns multiple contextual relationships in parallel.
- **Feed-Forward Network**: Performs nonlinear feature transformations.
- **Residual Connections & Normalization**: Support stable information flow.
- **Cross-Attention**: Connects decoder representations with encoded input.
- **Autoregressive Generation**: Produces output tokens sequentially.

Florence-2 interprets GUI screenshots and visual regions to generate semantic descriptions and task-specific outputs, supporting visual understanding in the OmniParser pipeline.



Transformer-Based Test Prioritization



1. Extracts 9 normalized features from each test case.
2. Features include risk, requirement relevance, coverage, evidence confidence, API relevance, and execution complexity.
3. Projects the 9-dimensional feature vector into a 64-dimensional representation.
4. Uses a 2-layer Transformer Encoder with 4 attention heads

5. Applies mean pooling to obtain the test representation.
6. Uses a regression head (64 → 32 → 1) to generate the priority score.
7. Produces a continuous test-priority score for each candidate test.
8. Tests are ranked according to their priority score.
9. Higher-priority tests are selected for earlier execution, supporting risk- and evidence-aware testing.

IMPACT AND BENEFIT

- **Reduced manual effort:** Automates evidence collection and supports quality analysis.
- **Comprehensive analysis:** Combines DOM, network, screenshot, and repository evidence.
- **Intelligent visual understanding:** Uses OmniParser and Transformer-based models to interpret web interfaces.
- **Modular architecture:** Uses specialized agents for different quality engineering tasks.
- **Improved defect analysis:** Consolidates evidence to support defect identification and testing.
- **Adaptability:** Allows the framework to be extended with additional agents and analysis capabilities.

CONCLUSION

The project presents a modular, multi-modal framework for web application analysis and quality engineering. It combines bounded crawling with comprehensive structural, operational, and visual data collection, while integrated vision and language components handle graphical interface detection, text recognition, and semantic interpretation to generate structured visual evidence. This fused evidence directly supports defect analysis, test generation, validation, and reporting. Furthermore, the modular architecture ensures high scalability, easier debugging, seamless component replacement, and flexible hardware execution.

REFERENCES

[1] C. Augusto, A. Bertolino, G. De Angelis, F. Lonetti, and J. Morán, **"Large Language Models for Software Testing: A Research Roadmap,"** arXiv preprint arXiv:2509.25043, 2026.

[2] M. Tasarsu, A. V. Tokmak, and C. Catal, **"Test Case Generation Using Large Language Models: A Systematic Literature Review,"** Cluster Computing, vol. 29, 2026, doi: 10.1007/s10586-026-06021-z.

[3] S. Alagarsamy, C. Tantithamthavorn, W. Takerngsaksiri, C. Arora, and A. Aleti, **"Enhancing Large Language Models for Text-to-Testcase Generation,"** Journal of Systems and Software, vol. 230, 112531, 2025. Paper

[4] Y. Lu, J. Yang, Y. Shen, and A. Awadallah, **"OmniParser for Pure Vision Based GUI Agent,"** arXiv preprint arXiv:2408.00203, 2024.

[5] P. Lewis et al., **"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,"** in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

Thank You